TOPIC 1 : INTRODUCTION

1. Given an array of strings words, return the first palindromic string in the array. If

there is no such string, return an empty string "". A string is palindromic if it reads

the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

AIM : To write a Python program to find and return the first palindromic string from a given array of strings. If no palindrome exists, return an empty string ("").

ALGORITHM :

1. Start.
2. Read the array of strings.
3. Traverse each string in the array.
4. Check whether the string is equal to its reverse.
5. If it is a palindrome, return that string and stop.
6. If no palindrome is found after checking all strings, return an empty string ("").
7. Stop

code:

```python
# Function to find the first palindromic string
def firstPalindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""

# User input
n = int(input("Enter the number of words: "))

words = []
print("Enter the words:")
for i in range(n):
    words.append(input())

# Find and display the first palindrome
result = firstPalindrome(words)

if result:
    print("First palindromic string:", result)
else:
    print("No palindromic string found.")
```

input and output:

```
Enter the number of words: 5
Enter the words:
abc
car
ada
racecar
cool
First palindromic string: ada
```

Time Complexity : O(n x m)

Space Complexity : O(1)

RESULT : The program successfully identifies and returns the first palindromic string from the given array of strings. If no palindrome is present, it returns an empty string ("").

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in

nums1 Return [answer1,answer2].

Example 1:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Example 2:

Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]

Output: [3,4]

Explanation:

The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So

answer1 is 3.

The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

AIM : To write a Python program to find the number of common elements between two integer arrays and return the counts as [answer1, answer2], where:

- answer1 = Number of elements in nums1 that exist in nums2.
- answer2 = Number of elements in nums2 that exist in n

ALGORITHM :

1. Start.
2. Read the two integer arrays nums1 and nums2.
3. Convert both arrays into sets for efficient searching.
4. Initialize answer1 and answer2 to 0.
5. Traverse each element in nums1.
6. If the element exists in nums2, increment answer1.
7. Traverse each element in nums2.
8. If the element exists in nums1, increment answer2.
9. Return [answer1, answer2].
10. Stop.

code:

```python
# Function to count matching elements
def findIntersectionValues(nums1, nums2):
    answer1 = 0
    answer2 = 0

    # Count elements of nums1 that exist in nums2
    for num in nums1:
        if num in nums2:
            answer1 += 1

    # Count elements of nums2 that exist in nums1
    for num in nums2:
        if num in nums1:
            answer2 += 1

    return [answer1, answer2]

# User input
n = int(input("Enter the size of nums1: "))
print("Enter the elements of nums1:")
nums1 = list(map(int, input().split(',')))

m = int(input("Enter the size of nums2: "))
print("Enter the elements of nums2:")
nums2 = list(map(int, input().split(',')))

# Check if the correct number of elements is entered
if len(nums1) != n or len(nums2) != m:
    print("Error: Number of elements does not match the specified size.")
else:
    result = findIntersectionValues(nums1, nums2)
    print("Output:", result)
```

input and output:

```
Enter the size of nums1: 3
Enter the elements of nums1:
2,3,2
Enter the size of nums2: 2
Enter the elements of nums2:
2,1
Output: [2, 1]
```

Time Complexity : O(n + m)

Space Complexity : O(n + m)

RESULT : The program successfully counts the number of elements in each array that are present in the other array and returns the result in the form [answer1, answer2].

3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarrayis a contiguous non-empty sequence of elements within an array.

Example 1:

Input: nums = [1,2,1]

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to 12 +12 + 12 + 22 + 22 + 22 = 15.

Example 2:

Input: nums = [1,1]

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to 12 +12 + 12 = 3.

AIM : To write a Python program to calculate the sum of the squares of the distinct counts of all possible subarrays of a given integer array.

ALGORITHM :

1. Start.
2. Read the integer array nums.
3. Initialize total_sum = 0.

4. For each starting index i:
   a. Create an empty set distinct.
   b. For each ending index j from ito the last element:
      i. Add nums[j] to the set.
      ii. Find the number of distinct elements using the size of the set.
      iii. Add the square of the distinct count to total_sum.
5. After processing all subarrays, display total_sum.
6. Stop.
7. **code:**

```python
# Function to calculate the sum of squares of distinct counts
def sumCounts(nums):
    total = 0
    n = len(nums)

    # Generate all subarrays
    for i in range(n):
        distinct = set()
        for j in range(i, n):
            distinct.add(nums[j])          # Add current element to the set
            count = len(distinct)          # Number of distinct elements
            total += count * count         # Add square of distinct count

    return total

# User input
n = int(input("Enter the size of the array: "))
print("Enter the array elements:")
nums = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(nums) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = sumCounts(nums)
    print("Output:", result)
```

input and output:

```
Enter the size of the array: 3
Enter the array elements:
1 2 1
Output: 15
```

Time Complexity: $O(n^2)$

Space Complexity: O(n) (for the set storing distinct elements)

Result： The program successfully computes the **sum of the squares of the distinct counts of all possible subarrays** in the given array and displays the correct result.

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k.

Example 1:

Input: nums = [3,1,2,2,2,1,3], k = 2

Output: 4

Explanation: There are 4 pairs that meet all the requirements:

nums[0] == nums[6], and 0 * 6 == 0, which is divisible by 2.

nums[2] == nums[3], and 2 * 3 == 6, which is divisible by 2.

nums[2] == nums[4], and 2 * 4 == 8, which is divisible by 2.

nums[3] == nums[4], and 3 * 4 == 12, which is divisible by 2.

Example 2:

Input: nums = [1,2,3,4], k = 1

Output: 0

Explanation: Since no value in nums is repeated, there are no pairs (i , j) that meet all the requirements.

Aim

To write a Python program to count the number of pairs (i, j) such that:

- $0 \leq i < j < n$
- nums[i] == nums[j]
- $(i \times j)$ is divisible by k.

Algorithm

1. Start.
2. Read the integer array nums and the integer k.
3. Initialize count = 0.
4. Traverse the array using two nested loops:
   - Outer loop from index i = 0 to n-1.
   - Inner loop from index j = i+1 to n-1.
5. For each pair (i, j):
   - Check if nums[i] == nums[j].
   - Check if $(i \times j)$ % k == 0.
   - If both conditions are true, increment count.
6. Display count.
7. Stop.

code :

```python
# Function to count valid pairs
def countPairs(nums, k):
    count = 0
    n = len(nums)

    # Check all possible pairs
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1

    return count

# User input
n = int(input("Enter the size of the array: "))
print("Enter the array elements:")
nums = list(map(int, input().split()))

k = int(input("Enter the value of k: "))

# Check if the correct number of elements is entered
if len(nums) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = countPairs(nums, k)
    print("Output:", result)
```

input and output:

```
Enter the size of the array: 7
Enter the array elements:
3 1 2 2 2 1 3
Enter the value of k: 2
Output: 4
```

Time Complexity: O(n²)

Space Complexity: O(1)

Result : The program successfully counts the number of pairs (i, j) where the elements are equal and the product of their indices is divisible by k, then displays the correct count.

5. Write a program FOR THE BELOW TEST CASES with least time complexity.

Test Cases: -

Input: {1, 2, 3, 4, 5} Expected Output: 5

Input: {7, 7, 7, 7, 7} Expected Output: 7

Input: {-10, 2, 3, -4, 5} Expected Output: 5

Aim

To write a Python program to find the **maximum element** in a given array using the least possible time complexity.

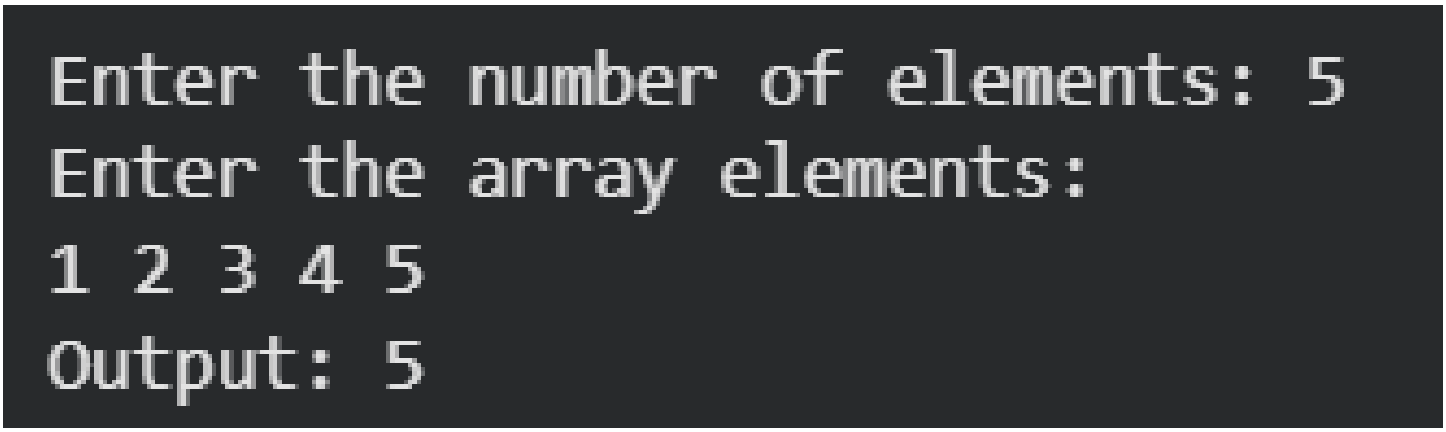
Algorithm

1. Start.
2. Read the array elements.
3. Assume the first element is the maximum.
4. Traverse the remaining elements of the array.
5. If the current element is greater than the maximum, update the maximum.
6. After traversing the array, display the maximum element.
7. Stop.

code :

```python
# Function to find the largest element
def findLargest(arr):
    largest = arr[0]

    for i in range(1, len(arr)):
        if arr[i] > largest:
            largest = arr[i]

    return largest

# User input
n = int(input("Enter the number of elements: "))

print("Enter the array elements:")
arr = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = findLargest(arr)
    print("Output:", result)
```

input and output :



Time Complexity: O(n)
Space Complexity: O(1)

Result : The program successfully finds and displays the maximum element in the given array by traversing it only once, achieving a time complexity of O(n) and a space complexity of O(1).

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

Empty List

Input: []

Expected Output: None or an appropriate message indicating

that the list is empty.

Single Element List

Input: [5]

Expected Output: 5

All Elements are the Same

Input: [3, 3, 3, 3, 3]

Expected Output: 3

Aim : To write a Python program to sort a list of numbers using an efficient sorting algorithm and then find the maximum element in the sorted list.

Algorithm

1. Start.
2. Read the list of numbers.
3. If the list is empty, display "List is empty" and stop.
4. Sort the list in ascending order using an efficient sorting algorithm.
5. The last element of the sorted list is the maximum element.
6. Display the maximum element.
7. Stop.

code :

```
# Function to sort the list and find the maximum element
def findMaximum(arr):
    if len(arr) == 0:
        return None

    arr.sort()          # Sort the list
    return arr[-1]      # Last element is the maximum

# User input
n = int(input("Enter the number of elements: "))

if n == 0:
    arr = []
else:
    print("Enter the array elements:")
    arr = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = findMaximum(arr)

    if result is None:
        print("The list is empty.")
    else:
        print("Sorted List:", arr)
        print("Maximum Element:", result)
```

input and output :

```
Enter the number of elements: 5
Enter the array elements:
1 2 3 4 5
Sorted List: [1, 2, 3, 4, 5]
Maximum Element: 5
```

- Time Complexity: O(n log n)
- Space Complexity: O(n) (assuming Python's built-in sorting is used).

Result

The program successfully sorts the given list using an efficient sorting algorithm and displays the **maximum element**. If the list is empty, it displays an appropriate message indicating that the list is empty.

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases:

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithmused) Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

Aim

To write a Python program to create a new list containing **only the unique elements** from a given list of numbers.

Algorithm

1. Start.
2. Read the input list of numbers.
3. Create an empty list called unique.
4. Traverse each element in the input list.
5. If the element is not present in unique, add it to unique.
6. After processing all elements, display the unique list.
7. Stop.

code :

```python
# Function to create a list of unique elements
def uniqueElements(arr):
    unique = []

    for num in arr:
        if num not in unique:
            unique.append(num)

    return unique

# User input
n = int(input("Enter the number of elements: "))

print("Enter the array elements:")
arr = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = uniqueElements(arr)
    print("Unique Elements:", result)
```

input and output :

```
Enter the number of elements: 8
Enter the array elements:
3 7 3 5 2 5 9 2
Unique Elements: [3, 7, 5, 2, 9]
```

Time Complexity: O(n²)

Space Complexity: O(n)

Result

The program successfully creates a new list containing only the **unique elements** from the given list while preserving their first occurrence.

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

Aim

To write a Python program to sort an array of integers using the **Bubble Sort** technique and analyze its time complexity using **Big-O notation**.

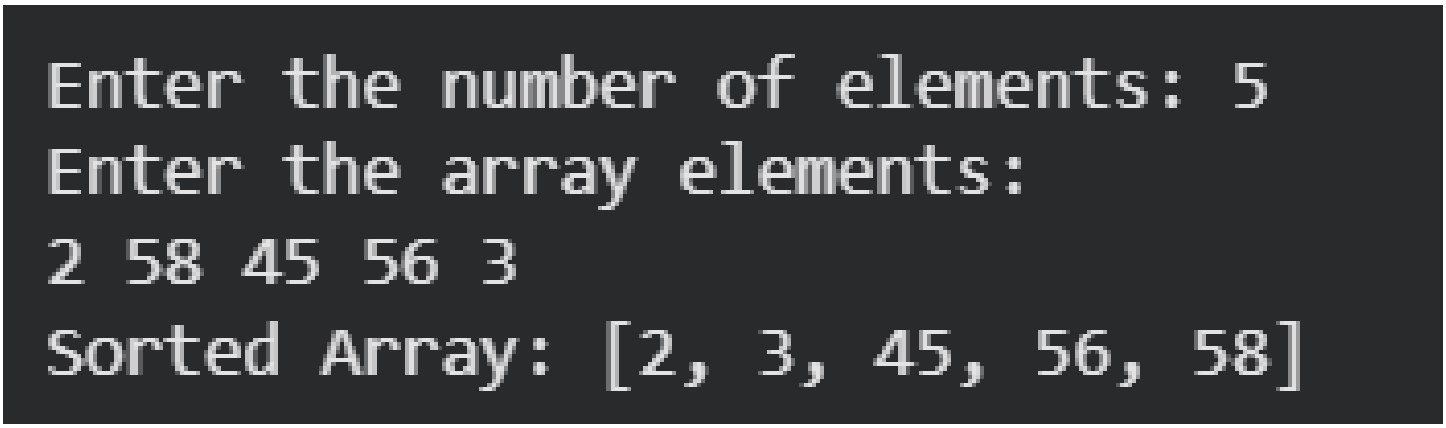
Algorithm

1. Start.
2. Read the array of integers.
3. Find the number of elements n.
4. Repeat n-1times:
   - Compare each pair of adjacent elements.
   - If the left element is greater than the right element, swap them.
5. After all passes, the array will be sorted in ascending order.
6. Display the sorted array.
7. Stop.

code :

```python
# Bubble Sort Program

def bubbleSort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap the elements
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

# User input
n = int(input("Enter the number of elements: "))

print("Enter the array elements:")
arr = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    sortedArray = bubbleSort(arr)
    print("Sorted Array:", sortedArray)
```

input and output :



Time Complexity: O(n²)

Space Complexity: O(1)

Result

The program successfully sorts the given array in **ascending order** using the **Bubble Sort** algorithm.

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30} KEY=100

Output : Element 100 is not found

Aim

To write a Python program to search for a given element in a **sorted array** using the **Binary Search** technique and analyze its time complexity using **Big-O notation**.

Algorithm

1. Start.
2. Read the sorted array and the key element.
3. Set low = 0 and high = n - 1.
4. Repeat while low <= high:
   - Find the middle index mid.
   - If arr[mid] equals the key, display its position and stop.
   - If the key is smaller than arr[mid], search the left half.
   - Otherwise, search the right half.
5. If the key is not found, display **"Element not found"**.
6. Stop.

code :

```python
# Binary Search Function
def binarySearch(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# User input
n = int(input("Enter the number of elements: "))

print("Enter the array elements:")
arr = list(map(int, input().split()))

key = int(input("Enter the key to search: "))

# Check if correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    arr.sort()   # Sort the array
    print("Sorted Array:", arr)

    position = binarySearch(arr, key)

    if position != -1:
```

input and output :

```
Enter the number of elements: 8
Enter the array elements:
3 4 6 -9 10 8 9 30
Enter the key to search: 10
Sorted Array: [-9, 3, 4, 6, 8, 9, 10, 30]
Element 10 is found at position 7
```

Time Complexity: O(log n)

Space Complexity: O(1)

Result

The program successfully searches for the given element using the Binary Search algorithm and displays whether the element is found or not.

10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

Aim

To write a Python program to sort an array of integers in **ascending order** using **Heap Sort** without using any built-in sorting functions.

Algorithm

1. Start.
2. Read the array of integers.
3. Build a max heap from the array.
4. Swap the root (maximum element) with the last element.
5. Reduce the heap size by one.
6. Heapify the root to restore the max heap.
7. Repeat Steps 4–6 until the heap size becomes 1.
8. Display the sorted array.
9. Stop.

code :

```python
# Merge Sort Program

def mergeSort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2

        left = arr[:mid]
        right = arr[mid:]

        mergeSort(left)
        mergeSort(right)

        i = j = k = 0

        # Merge the two halves
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        # Copy remaining elements of left half
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
```

```
        # Copy remaining elements of right half
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

# User input
n = int(input("Enter the number of elements: "))

print("Enter the array elements:")
nums = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(nums) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    mergeSort(nums)
    print("Sorted Array:", nums)
```

input and output :

```
Enter the number of elements: 5
Enter the array elements:
2 7 5 7 5
Sorted Array: [2, 5, 5, 7, 7]
```

- **Time Complexity: O(n log n)**
- **Space Complexity: O(1)**

Result

The program successfully sorts the given array in ascending order using the Heap Sort algorithm without using any built-in sorting functions.

11. Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

· Input: m=2,n=2,N=2,i=0,j=0 · Output: 6

· Input: m=1,n=3,N=3,i=0,j=1 · Output: 12

Aim

To write a Python program to find the **number of ways to move a ball out of an m × n grid boundary in exactly N steps** from a given starting cell.

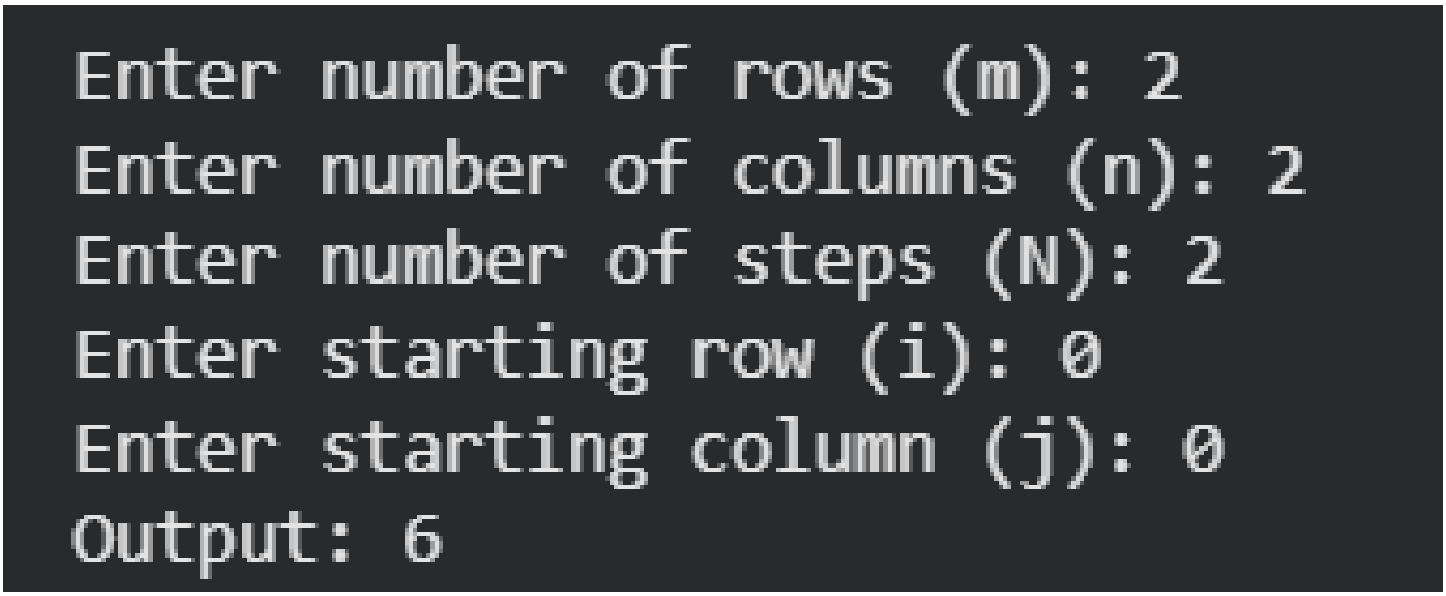
Algorithm

1. Start.
2. Read the values of m, n, N, starting row i, and starting column j.
3. If the ball moves outside the grid before or at the Nth step, count it as one valid way.
4. If no steps are left and the ball is still inside the grid, return 0.
5. From the current cell, recursively move the ball in four directions:
   - Up
   - Down
   - Left
   - Right
6. Sum the number of valid ways obtained from all four directions.
7. Display the total number of ways.
8. Stop.

code :

```python
# Function to count the number of ways
def findWays(m, n, N, i, j):
    # Ball moves out of the grid
    if i < 0 or i >= m or j < 0 or j >= n:
        return 1

    # No steps remaining
    if N == 0:
        return 0

    # Move in four directions
    up = findWays(m, n, N - 1, i - 1, j)
    down = findWays(m, n, N - 1, i + 1, j)
    left = findWays(m, n, N - 1, i, j - 1)
    right = findWays(m, n, N - 1, i, j + 1)

    return up + down + left + right

# User input
m = int(input("Enter number of rows (m): "))
n = int(input("Enter number of columns (n): "))
N = int(input("Enter number of steps (N): "))
i = int(input("Enter starting row (i): "))
j = int(input("Enter starting column (j): "))

result = findWays(m, n, N, i, j)

print("Output:", result)
```

input and output :



Time Complexity: $O(4^N)$

Space Complexity: O(N)

Result

The program successfully calculates the number of ways to move the ball outside the grid boundary in exactly N steps from the given starting position.

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii)Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

Aim

To write a Python program to find the **maximum amount of money that can be robbed from houses arranged in a circle** without robbing two adjacent houses.

Algorithm

1. Start.
2. Read the array nums representing the money in each house.
3. If there is only one house, return its value.
4. Since the houses are arranged in a circle:
   - Find the maximum money by robbing houses from index 0 to n-2.
   - Find the maximum money by robbing houses from index 1 to n-1.
5. Use Dynamic Programming for each case:
   - At each house, choose the maximum of:
     - Rob the current house and add the value from two houses before.
     - Skip the current house.
6. Return the maximum of the two cases.
7. Display the result.
8. Stop.

code :

```python
# Function to rob houses in a straight line
def robLinear(houses):
    prev1 = 0
    prev2 = 0

    for money in houses:
        current = max(prev1, prev2 + money)
        prev2 = prev1
        prev1 = current

    return prev1

# Function to rob houses in a circle
def robCircular(nums):
    n = len(nums)

    if n == 0:
        return 0
    if n == 1:
        return nums[0]

    # Either rob from house 0 to n-2 or from house 1 to n-1
    return max(robLinear(nums[:-1]), robLinear(nums[1:]))

# User input
n = int(input("Enter the number of houses: "))

print("Enter the money in each house:")
nums = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(nums) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = robCircular(nums)
```

input and output :

```
Enter the number of houses: 3
Enter the money in each house:
2 3 2
Maximum money that can be robbed: 3
```

Time Complexity: O(n)

Space Complexity: O(1)

Result

The program successfully finds the maximum amount of money that can be robbed from circularly arranged houses without robbing two adjacent houses.

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

Input: n=4 Output: 5

Input: n=3 Output: 3

Aim

To write a Python program to find the **number of distinct ways to climb a staircase** of n steps, where at each step you can climb either **1** or **2** steps.

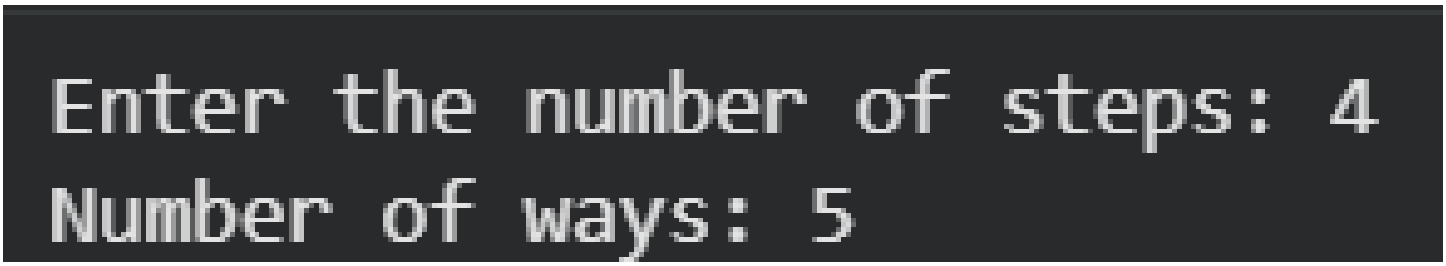
Algorithm

1. Start.
2. Read the value of n.
3. If n is 1, return 1.
4. If n is 2, return 2.
5. Initialize:
   - first = 1
   - second = 2
6. For each step from 3 to n:
   - current = first + second
   - Update first = second
   - Update second = current
7. Display second as the total number of ways.
8. Stop.

code :

```python
# Function to find the number of ways
def climbStairs(n):
    if n == 1:
        return 1
    if n == 2:
        return 2

    first = 1
    second = 2

    for i in range(3, n + 1):
        third = first + second
        first = second
        second = third

    return second

# User input
n = int(input("Enter the number of steps: "))

result = climbStairs(n)

print("Number of ways:", result)
```

input and output :



Time Complexity: O(n)

Space Complexity: O(1)

Result

The program successfully calculates the **number of distinct ways** to climb a staircase of n steps when either **1** or **2** steps can be taken at a time.

14. A robot is located at the top-left corner of a m×n grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

Input: m=7,n=3 Output: 28

Input: m=3,n=2 Output: 3

Aim

To write a Python program to find the **number of unique paths** for a robot to move from the **top-left corner** to the **bottom-right corner** of an m × n grid, where the robot can move only **right** or **down**.

Algorithm

1. Start.
2. Read the values of m and n.
3. Create a 2D array dp of size m × n.
4. Initialize the first row and first column with 1, since there is only one way to reach those cells.
5. For each remaining cell:
   - Set dp[i][j] = dp[i-1][j] + dp[i][j-1].
6. The value at dp[m-1][n-1] gives the total number of unique paths.
7. Display the result.
8. Stop.

code :

```python
# Function to find the number of unique paths
def uniquePaths(m, n):
    dp = [[1 for j in range(n)] for i in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

    return dp[m - 1][n - 1]

# User input
m = int(input("Enter the number of rows (m): "))
n = int(input("Enter the number of columns (n): "))

result = uniquePaths(m, n)

print("Number of unique paths:", result)
```

input and output :

```
Enter the number of rows (m): 7
Enter the number of columns (n): 3
Number of unique paths: 28
```

Time Complexity: O(m × n)

Space Complexity: O(m × n)

Result

The program successfully calculates the number of unique paths from the top-left corner to the bottom-right corner of the grid by moving only right or down.

15. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1:

Input: s = "abbxxxxzzy"

Output: [[3,6]]

Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2:

Input: s = "abc"

Output: []

Explanation: We have groups "a", "b", and "c", none of which are large groups.

Aim

To write a Python program to find the **intervals of all large groups** (groups containing **3 or more consecutive identical characters**) in a given string.

Algorithm

1. Start.
2. Read the input string s.
3. Initialize an empty list result.
4. Traverse the string using an index start to mark the beginning of each group.
5. Move another index until a different character is found.
6. If the group length is 3 or more, store its start and end indices in result.
7. Continue until the end of the string.
8. Display the list of intervals.
9. Stop.

code :

```python
# Function to find large groups
def largeGroupPositions(s):
    result = []
    n = len(s)
    i = 0

    while i < n:
        start = i

        while i < n - 1 and s[i] == s[i + 1]:
            i += 1

        end = i

        if end - start + 1 >= 3:
            result.append([start, end])

        i += 1

    return result

# User input
s = input("Enter the string: ")

result = largeGroupPositions(s)

print("Output:", result)
```

input and output :

```
Enter the string: abbxxxxzzy
Output: [[3, 6]]
```

Time Complexity: O(n)

Space Complexity: O(k)

Result

The program successfully identifies all **large groups** (groups of three or more consecutive identical characters) and returns their **start and end indices** in increasing order.

15. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules Any live cell with fewer than two live neighbors dies as if caused by under-population. Any live cell with two or three live neighbors lives on to the next generation. Any live cell with more than three live neighbors dies, as if by over-population. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction. The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return the next state.

Example 1:

Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]

Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

Example 2:

Input: board = [[1,1],[1,0]]

Output: [[1,1],[1,1]]

Aim

To write a Python program to compute the **next state** of the **Game of Life** based on the given rules for live and dead cells.

Algorithm

1. Start.
2. Read the m × n grid board.
3. Create a copy of the board to store the next state.
4. For each cell in the board:
   - Count the number of live neighbors among its eight neighboring cells.
   - Apply the Game of Life rules:
     - If the cell is live and has fewer than 2 or more than 3 live neighbors, make it dead.
     - If the cell is live and has 2 or 3 live neighbors, keep it alive.
     - If the cell is dead and has exactly 3 live neighbors, make it alive.
5. After updating all cells, display the new board.
6. Stop.

code :

```python
# Function to generate the next state
def gameOfLife(board):
    rows = len(board)
    cols = len(board[0])

    # Copy of the original board
    newBoard = [row[:] for row in board]

    # Directions of the 8 neighbors
    directions = [(-1,-1), (-1,0), (-1,1),
                  (0,-1),          (0,1),
                  (1,-1), (1,0), (1,1)]

    for i in range(rows):
        for j in range(cols):
            liveNeighbors = 0

            # Count live neighbors
            for dr, dc in directions:
                r = i + dr
                c = j + dc

                if 0 <= r < rows and 0 <= c < cols:
                    if board[r][c] == 1:
                        liveNeighbors += 1

            # Apply Game of Life rules
            if board[i][j] == 1:
                if liveNeighbors < 2 or liveNeighbors > 3:
                    newBoard[i][j] = 0
            else:
                if liveNeighbors == 3:
                    newBoard[i][j] = 1
```

```python
    return newBoard

# User input
m = int(input("Enter the number of rows: "))
n = int(input("Enter the number of columns: "))

print("Enter the board (0 for dead, 1 for live):")
board = []

for i in range(m):
    row = list(map(int, input().split()))
    board.append(row)

# Display next state
result = gameOfLife(board)

print("Next State:")
for row in result:
    print(row)
```

input and output :

```
Enter the number of rows: 4
Enter the number of columns: 3
Enter the board (0 for dead, 1 for live):
0 1 0
0 0 1
1 1 1
0 0 0
Next State:
[0, 0, 0]
[1, 0, 1]
[0, 1, 1]
[0, 1, 0]
```

Time Complexity: O(m × n)

Space Complexity: O(m × n)

Result

The program successfully computes the **next generation** of the Game of Life by applying the four rules simultaneously to every cell in the grid.

16. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.

Now after pouring some non-negative integer cups of champagne, return how full the jth glass in the ith row is (both i and j are 0 indexed.)

Example 1:

Input: poured = 1, query_row = 1, query_glass = 1

Output: 0.00000

Explanation: We poured 1 cup of champange to the top glass of the

tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.

Example 2:

Input: poured = 2, query_row = 1, query_glass = 1

Output: 0.50000

Explanation: We poured 2 cups of champange to the top glass of the

tower (which is indexed as (0, 0)). There is one cup of excess liquid.

The glass indexed as (1, 0) and the glass indexed as (1, 1) will share

the excess liquid equally, and each will get half cup of champange.

Aim

To write a Python program to determine **how full a specific glass is** after pouring a given number of cups of champagne into the top glass of a champagne tower.

Algorithm

1. Start.
2. Read the values of poured, query_row, and query_glass.
3. Create a 2D array to represent the champagne tower.
4. Pour all the champagne into the top glass (0,0).
5. For each row up to query_row:
   - If a glass contains more than 1 cup:
     - Compute the excess = (current amount − 1) / 2.
     - Pour the excess equally into the two glasses below.
     - Keep only 1 cup in the current glass.
6. After processing all rows, the required glass fullness is the minimum of 1 and the amount in the queried glass.
7. Display the result.
8. Stop.

code :

```python
# Function to find how full the queried glass is
def champagneTower(poured, query_row, query_glass):
    # Create a 2D array for the tower
    dp = [[0.0] * (query_row + 2) for _ in range(query_row + 2)]

    dp[0][0] = poured

    # Simulate the flow of champagne
    for i in range(query_row + 1):
        for j in range(i + 1):
            if dp[i][j] > 1:
                excess = (dp[i][j] - 1) / 2
                dp[i + 1][j] += excess
                dp[i + 1][j + 1] += excess
                dp[i][j] = 1

    return dp[query_row][query_glass]

# User input
poured = int(input("Enter the number of cups poured: "))
query_row = int(input("Enter the query row: "))
query_glass = int(input("Enter the query glass: "))

result = champagneTower(poured, query_row, query_glass)

print("Output: {:.5f}".format(result))
```

input and output :

```
Enter the number of cups poured: 2
Enter the query row: 1
Enter the query glass: 1
Output: 0.50000
```

input and output :

Time Complexity: O(query_row²)

Space Complexity: O(query_row²)

Result

The program successfully computes the amount of champagne in the specified glass after the given number of cups has been poured into the champagne tower.